REDES DE COMUNICAÇÃO

Licenciatura em Engenharia Informática



FACULDADE DE CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D COIMBRA

Cristiano David Nascimento Domingues Santos nº2022219696
David Cameijo Pinheiro nº2021246865

EXPLICAÇÃO DO CÓDIGO E DA ESTRUTURA

“dados.h”

Começando pelo header file “**dados.h**”, é o ficheiro onde são definidos tudo o que são tamanhos de buffers, onde é também definido o “UDP_MULTICAST_PORT”, e onde são criadas várias “structs”, a estrutura para os utilizadores, para as turmas, da memória partilhada e dos argumentos das threads.
Além disso, também são aqui declaradas todas as funções dos “comandos_cliente.c” e do“comandos_admin.c”

“comandos_cliente.c”

Neste ficheiro, procuramos criar todas as funções necessárias para que tanto o professor como o aluno, conseguissem realizar todas as funcionalidades desejadas. Começando pela função “**list_classes**”, A função verifica se não há turmas no sistema, se houver turmas, a função cria uma mensagem inicial para começar a listar as turmas.
De seguida, itera sobre cada turma, adicionando informações sobre cada uma à mensagem de resposta.
Verifica se a adição de uma nova turma à lista não ultrapassa o tamanho máximo permitido do buffer (BUFLEN). Se ultrapassar, a adição é interrompida.

Na função “**list_subscribed**”, a função procura pelo utilizador específico na lista de utilizadores, se o utilizador for encontrado, o seu índice é armazenado na variável “user_index”.

Após localizar o utilizador, a função verifica se ele não está inscrito em nenhuma turma. A função inicia a construção da mensagem de resposta, itera sobre cada classe à qual o utilizador está inscrito, escrevendo informações sobre cada uma delas. Após verificações já descritas anteriormente, a função envia a

mensagem ao cliente através do socket. Na função "**subscribe_user_to_class**", vamos procurar o utilizador e a turma, verificar se a turma tem capacidade para receber o aluno e se o aluno ainda não está inscrito na turma, e de seguida, vamos adicioná-lo a essa mesma turma. Já a função "***get_class_ip**", é baseada na memória partilhada, apenas vamos assumir que a "shared memory" contém a lista de turmas e os seus ip`s correspondentes e vamos procurar o que desejamos, caso ele exista. A maior função deste ficheiro é a "**create_class**", começamos por verificar se a turma já existe, depois se não existir adicionamos a turma, geramos um novo endereço "multicast", criamos os sockets, definimos o "multicast_adress" e a porta, e por fim, adicionamos a turma à memória partilhada. Para terminar, temos a função "**send_content_to_class**", que vai procurar a turma, preparar a mensagem a enviar, e enviar essa mesma mensagem.

"comandos_admin.c"

Neste ficheiro, procuramos criar todas as funções necessárias para que o administrador conseguisse realizar todas as funcionalidades desejadas. Começando pela função "**show_menu**", é apenas algo estético para que ao dar login como "admin" apareça um menu com as funcionalidades possíveis. De seguida, temos a função "**add_user**", que como o próprio nome indica, usa-se para adicionar um novo utilizador, seja qual for a sua função. Vamos verificar se esse utilizador já existe, vamos adicioná-lo caso não exista e vamos enviar a mensagem de confirmação para que se saiba que foi realmente adicionado com sucesso. Partindo para a função "**add_user_to_config_file**", basicamente, vamos ver se o ficheiro de texto tem alguma linha em branco antes do seu final, se tiver, adicionamos aí o novo utilizador, senão, adicionamos no fim do ficheiro. Já a função "**remove_user**", é usada para remover um utilizador, a função percorre a lista de utilizadores na memória partilhada, se encontrar o utilizador, marca "user_found" como 1 e remove o utilizador deslocando os elementos do "array" para preencher o espaço

deixado, por fim diminui o contador de utilizadores. A função “**update_config_file**”, serve apenas para escrever o desejado no ficheiro. Para terminar, temos as funções “**admin_login**”, para o administrador conseguir entrar, a função “**shutdown_server**”, para fechar o servidor e a função “**list_users**”, que vai verificar se existem utilizadores e se existirem, vai apresentá-los sob uma forma de lista.

“serverFinal.c”

Começamos por inicializar todas as variáveis necessárias, após isto temos as funções “**destroy_memory_share**” e a função “**handle_sigint**”, a primeira para libertar a memória, e a segunda para terminar as ligações e fechar o servidor. De seguida, temos a “**main**”, onde vamos começar por verificar as portas, depois definir o semáforo para controlo da memória partilhada, criar essa mesma memória partilhada, e criar as “**threads**” para o “**udp**” e para o “**tcp**”, de forma que as ligações sejam estabelecidas e as mensagens possam ser transmitidas. Temos ainda neste ficheiro as funções “***udp**” e “***tcp**”, que são responsáveis pelas respetivas ligações, algo que já tinha sido feito e explicado para a primeira meta. Temos ainda as funções “**receive_client_command**” e “**receive_admin_command**”, que como o próprio nome indica, vão verificar tudo o que o “cliente” e o admin tentarem fazer, isto é, consoante o que for introduzido pelo cliente e pelo admin, correspondentemente, estas funções vão verificar se tudo foi feito de forma correta, se não existem problemas de escrita, e se tudo estiver bem feito, irão realizar as tarefas correspondentes a cada comando introduzido.

Temos, ainda, uma função para a criação da “shared memory”, uma função para dar print aquando da existência de erros, uma função do tipo “bool” para verificar o utilizador e para verificar o seu tipo e temos outra função que serve apenas para abrir o ficheiro para leitura.

“cliente_final.c”

Começamos com algumas definições, criamos a struct “MulticastArgs”, e entramos de seguida na “**main**”. Na main, definimos a struct para o servidor e a struct para guardar infomações do host, verifica se a quantidade de argumentos é diferente de 3 (nome do programa, host, porta), se for, exibe uma mensagem de erro e termina a execução. De seguida, é criada a thread para ser possível receber mensagens do servidor, e por fim é fechado o socket. Temos mais uma vez a função “**erro**” para dar print aos erros possíveis. Na função “**receive_multicast**”, vamos criar o “udp socket”, introduzir o código que vai permitir que vários sockets usem a mesma porta, e de seguida vamos configurar a estrutura de endereços multicast. Começamos por usar a porta para o grupo multicast, dar “bind” a essa mesma porta, especificar qual o grupo multicast, e dar “join” a esse grupo, de seguida, estamos prontos para receber mensagens desse mesmo grupo multicast. Temos ainda a função “**receive_messages**”, a função “receive_messages” é uma thread que continuamente recebe e processa mensagens de um servidor TCP. Ela lida com erros de receção e o encerramento da conexão, processa mensagens recebidas e cria uma thread para tratar mensagens multicast quando necessário. Isto permite que o programa principal continue a executar outras tarefas enquanto a thread gere a comunicação de rede. Por fim, temos a função “**get_last_octet**”, a função “get_last_octet” processa um endereço IP fornecido como string, dividindo-o nos seus componentes. Ela faz uma cópia da string original, divide-a utilizando '.' como delimitador, armazena o último octeto encontrado, e converte-o para um inteiro, que é então retornado. Esta função é útil para extrair partes específicas de um endereço IP, especialmente quando apenas o último octeto é necessário para outras operações.